

Experiment 12

Selenium Page Object Model with Behave

Python BDD RESTful Automations

Overview`

This experiment shows how to organize a Selenium browser test with Behave and the Page Object Model (POM). The example automates the SauceDemo login flow. The implementation keeps locators and page actions in one class, while the Gherkin scenario and step definitions describe the behavior in plain language.

Problem and Objective

Putting Selenium locators directly into many Behave steps makes an automation suite difficult to maintain. A small application change can then require edits in several files. The objective is to build and run a working BDD test while separating browser setup, page interaction, and verification responsibilities.

Tools and Concepts

Item	Purpose
Python 3	Runs the automation code
Selenium WebDriver	Controls the Chrome browser
Behave and Gherkin	Expresses BDD scenarios and steps
Page Object Model	Centralizes locators and page actions
Object-oriented programming	Encapsulates the login page behavior

Project Structure

Create the following structure in a new VS Code folder:

```
requirements.txt
behave.ini
pages/login_page.py
features/pom_login.feature
features/environment.py
features/steps/pom_steps.py
```

Implementation

Page Object

The LoginPage class owns the page URL, element locators, explicit waits, and login operation. If an HTML ID changes, the locator can be updated here without rewriting the scenario wording.

Feature and Steps

The feature file describes the expected behavior: load the login page, sign in with the standard account, and confirm that the inventory dashboard appears. The step definitions create the driver through the Behave environment hook, call the page object's methods, and verify the resulting URL.

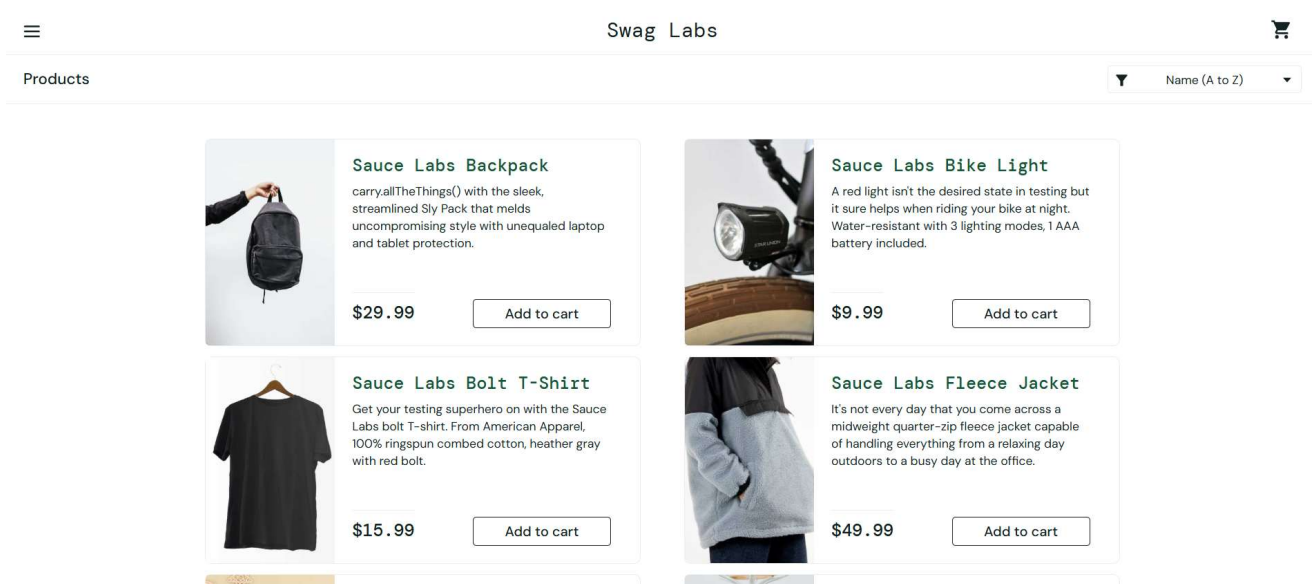
How to Run in VS Code

1. Open the project folder in VS Code.
2. Create and activate a virtual environment: `python -m venv .venv` followed by `.\.venv\Scripts\Activate.ps1`.
3. Install dependencies with `python -m pip install -r requirements.txt`.
4. Run the scenario with `python -m behave`.
5. Use `HEADLESS=1` before the command when a visible browser is not required.

Google Chrome must be installed. Selenium Manager normally obtains the compatible driver automatically. A successful run produces one passed feature, one passed scenario, and three passed steps. The test also saves `pom_success.png` after the dashboard is reached.

```
Feature: Page Object Model integration # features/pom_login.feature:1
  As a QA engineer
  I want login interactions kept inside a page object
  So that UI locator changes have one maintenance point
  Scenario: A standard user opens the inventory dashboard # features/pom_login.feature:6
    Given the login page is loaded # features/steps/pom_steps.py:8
    When the user signs in with the standard account # features/steps/pom_steps.py:14
    Then the inventory dashboard is displayed # features/steps/pom_steps.py:19

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped, 0 undefined
Took 0m1.460s
```



Result and Observation

The test separates concerns cleanly: `environment.py` manages the browser lifecycle, `login_page.py` contains UI knowledge, and `pom_steps.py` coordinates readable behavior and assertions. The step definitions do not contain raw Selenium locators. This structure reduces duplication and makes future UI maintenance more focused.

Conclusion

Combining Behave with the Page Object Model creates a readable and maintainable browser-automation foundation. The scenario communicates the user journey, while the page object provides a single maintenance point for the page's technical details.